

Contents

Pet Store Agentic – Detailed Design	1
The BaseAdapter contract (existing framework ABB)	1
Precedent: CrewAIOrchestratorAdapter (existing, elsewhere in the framework – not part of this project)	1
DiagnosisAgent – the ABB contract (new, this project)	2
SdkDiagnosisAgent – SBB-A (new, this project)	3
DirectApiDiagnosisAgent – SBB-B (new, this project)	3
Config binding – selecting the SBB	3
GateRegistry – Provider Adapter Pattern (minimal now, extensible later)	4
Open design questions (resolve before Phase 4 implementation)	4

Pet Store Agentic – Detailed Design

Class-level contracts referenced conceptually in [Architecture_Guide.md](#). That file explains *why* these shapes exist; this file is the exact *what* – signatures, not narrative. See `project.md` for the full build spec these contracts sit inside.

Nothing here is implemented yet – this is the design surface to build against once Phase 3/4 (see `PLAN.md`) starts.

The BaseAdapter contract (existing framework ABB)

`k9_aif_abb/k9_core/base_adapter.py` – already shipped, not new:

```
class BaseAdapter(ABC):
    def __init__(self, adapter_name: str | None = None, metadata: dict | None = None): ...
    def validate_payload(self, payload: dict) -> None: ...           # optional hook
    @abstractmethod
    def adapt_input(self, payload: dict) -> dict: ...
    @abstractmethod
    def adapt_output(self, result: Any) -> dict: ...
    def get_adapter_metadata(self) -> dict: ...
```

Precedent: CrewAIOrchestratorAdapter (existing, elsewhere in the framework – not part of this project)

`k9_aif_abb/k9_adapters/crewai/crewai_orchestrator_adapter.py`. Cited only to establish the pattern being followed – **Pet Store Agentic has no CrewAI dependency.**

```
class CrewAIOrchestratorAdapter(BaseOrchestrator, BaseAdapter):

    def __init__(self, crew: Any, name: str | None = None) -> None:
        BaseAdapter.__init__(self, adapter_name=name or "CrewAIOrchestratorAdapter")
        BaseOrchestrator.__init__(self)
        self.crew = crew

    def adapt_input(self, payload: dict) -> dict:
        # K9-AIF payload -> CrewAI's expected shape
        return {
```

```

        "message": payload.get("message") or payload.get("input") or payload.get("query") or "",
        "intent": payload.get("intent"),
        "context": payload.get("context", {}),
        "metadata": payload.get("metadata", {}),
        "raw_payload": payload,
    }

    def adapt_output(self, result: Any) -> dict:
        # CrewAI's result -> K9-AIF's expected shape
        if isinstance(result, dict):
            return {"status": "success", "result": result,
                    "output_text": result.get("output") or result.get("message") or str(result)}
        return {"status": "success", "result": result, "output_text": str(result)}

    def execute_flow(self, payload: dict) -> dict:
        self.validate_payload(payload)
        crew_input = self.adapt_input(payload)
        if hasattr(self.crew, "kickoff"):
            result = self.crew.kickoff(inputs=crew_input)
        elif hasattr(self.crew, "run"):
            result = self.crew.run(crew_input)
        else:
            raise AttributeError("Provided CrewAI crew does not support kickoff() or run().")
        return self.adapt_output(result)

```

Wraps at the **Orchestrator** layer because Crew is itself a multi-agent construct with its own kick-off()/run().

DiagnosisAgent – the ABB contract (new, this project)

```

@dataclass(frozen=True)
class DiagnosisRequest:
    narrative: str
    tank_profile: TankProfile | None
    customer_id: str

@dataclass(frozen=True)
class DiagnosisResult:
    likely_causes: list[Cause]
    recommended_products: list[SKU]
    confidence: Confidence
    escalate_to_human: bool
    provenance: ProvenanceChain

class DiagnosisAgent(BaseAgent):
    @abstractmethod
    async def diagnose(self, request: DiagnosisRequest) -> DiagnosisResult: ...

```

SdkDiagnosisAgent – SBB-A (new, this project)

Wraps the Claude Agent SDK at the **Agent** layer – the SDK’s unit of encapsulation (one session, one autonomous tool loop) sits one layer below CrewAI’s Crew, so this is where BaseAdapter attaches:

```
class SdkDiagnosisAgent(DiagnosisAgent, BaseAdapter):

    def __init__(self, config: dict | None = None, **kwargs):
        DiagnosisAgent.__init__(self, config or {}, **kwargs)
        BaseAdapter.__init__(self, adapter_name="SdkDiagnosisAgent")
        self._sdk_session = None  # constructed per-request or reused – TBD at implementation time

    def adapt_input(self, request: DiagnosisRequest) -> dict:
        # DiagnosisRequest -> whatever the SDK session's message/tool-context shape expects
        ...

    def adapt_output(self, sdk_result: Any) -> DiagnosisResult:
        # SDK's raw session output -> DiagnosisResult
        ...

    async def diagnose(self, request: DiagnosisRequest) -> DiagnosisResult:
        sdk_input = self.adapt_input(request)
        sdk_result = await self._run_sdk_session(sdk_input)  # multi-turn, tools, PreToolUse gate hook
        return self.adapt_output(sdk_result)
```

Tools exposed to the SDK session: search_catalog, check_species_compatibility, retrieve_care_guide, check_inventory. Subagent spawning disabled (see Architecture_Guide.md §2). Gate enforced via PreToolUse hook (see sequence diagram, diagrams/03-gate-enforcement-sequence.puml). Does **not** go through llm_invoke/K9ModelRouter – the SDK owns its own inference loop; document this explicitly wherever this class is described, since it’s the one exception to standard K9-AIF agent convention in this project.

DirectApiDiagnosisAgent – SBB-B (new, this project)

No BaseAdapter needed here – a direct Messages API call isn’t “bridging an external agent framework” in the same sense; it’s simply K9-AIF’s own inference path (llm_invoke) applied with explicit, code-driven tool orchestration instead of an autonomous loop:

```
class DirectApiDiagnosisAgent(DiagnosisAgent):

    async def diagnose(self, request: DiagnosisRequest) -> DiagnosisResult:
        # Single-shot structured call via llm_invoke/K9ModelRouter.
        # Tool use (catalog search, compatibility check, etc.) orchestrated
        # explicitly in this method – the code decides what to call next,
        # not the model. Livestock gate is an explicit branch here, not a hook.
        ...
```

Config binding – selecting the SBB

```
# config/sbb_bindings.yaml
diagnosis_abb: sdk      # or: direct_api
```

Both `SdkDiagnosisAgent` and `DirectApiDiagnosisAgent` satisfy `DiagnosisAgent` identically from the Squad's perspective – see `diagrams/02-abb-sbb-class.puml`.

GateRegistry – Provider Adapter Pattern (minimal now, extensible later)

`project.md §7` requires gate state to live in an external registry the `PreToolUse` hook can query directly – but doesn't specify who actually flips a gate from `PENDING` to `APPROVED`. In a real deployment that's a human, reviewing the order in some queue. This project gives that concern the same three-layer treatment every other infrastructure concern in K9-AIF gets (see the framework's Provider Adapter Pattern – Secret Management, Cache, Object Storage all follow this shape):

```
# k9_core/gates/base_gate_registry.py – ABB contract
class BaseGateRegistry(ABC):
    @abstractmethod
    async def status(self, order_id: str, gate_type: GateType) -> Gate: ...
    @abstractmethod
    async def request_approval(self, order_id: str, gate_type: GateType, context: dict) -> Gate: ...
    @abstractmethod
    async def resolve(self, gate_id: str, approved: bool, approver: str) -> Gate: ...
```

Phase 5 (this build): SimpleGateRegistry. SQLite-backed, no external dependencies, no queue UI. `resolve()` is called directly – by a test, by a CLI command, or by a minimal local endpoint. This is enough to prove `test_gate_cannot_be_bypassed.py` and `test_gate_survives_compaction.py`: both only care that the hook queries external state and gets a real answer, not about how that state was set.

Later phase (not this build): K9xHilGateRegistryAdapter. Backs the same `BaseGateRegistry` contract with a real K9x HIL Project -> App -> Queue -> Task flow – a human reviewer sees the pending live-stock order in an actual queue and approves or rejects it there, and that action is what calls `resolve()`. `PreToolUse` hook code, `SdkDiagnosisAgent`, `DirectApiDiagnosisAgent`, and both adversarial tests stay completely unchanged – only `config/gates.yaml`'s `provider:` key changes. This is the same substrate-neutrality claim the project already makes for diagnosis (§ above), applied to who approves the gate instead of what performs the diagnosis.

```
# config/gates.yaml
gates:
  provider: simple          # simple (default, this build) | k9x_hil (future)
```

Deliberately **not building the k9x_hil adapter now** – matches this project's own Anti-Goal against adding infrastructure the thesis doesn't require yet. The point of designing the contract this way now is that adding it later is a new adapter file and a config change, not a redesign.

Open design questions (resolve before Phase 4 implementation)

- Exact `PreToolUse` hook payload shape and deny contract – verify against the installed `claude-agent-sdk`, don't assume (see `project.md §0`, `PLAN.md`).
- Whether `_sdk_session` is constructed per-request or held across requests within a `DiagnosisSession` – affects `GateRegistry` query timing.
- Exact `adapt_input/adapt_output` field mapping for `SdkDiagnosisAgent` – left as ... above deliberately; fill in once the SDK's actual session/message API is confirmed.